COMPILER DESIGN — MINI PROJECT (REVIEW 3)

Source Code Metrics Analyzer
Implementation, Results and Demonstration

Assessment 10 | Submission Due: 21 October 2026

Name	Buvan Mahendran
Registration Number	24BCE2716
VIT Email ID	buvan.punithavathi2024@vitstudent.ac.in

Assessment 10 submission contents: source code, results, demonstration screenshots, and public GitHub repository link.

1. Abstract

Source Code Metrics Analyzer is a Python-based static analysis tool that processes Python source files through lexical analysis, syntax analysis, semantic analysis using a symbol table, and metrics computation. It reports token categories, source-code structure, and selected software-quality metrics without executing the analyzed program.

2. Project Objectives

• Tokenize and classify keywords, identifiers, operators, literals, and comments.

• Parse input using Python AST facilities.

• Build a symbol table and flag unresolved identifier references.

• Calculate LOC, cyclomatic complexity, maximum nesting depth, function count, and comment density.

• Display a readable report and save a JSON metrics report.

3. Implementation and Methodology

Phase 1 — Lexical analysis: tokenize classifies tokens into keywords, identifiers, operators, literals, comments, and other.

Phase 2 — Syntax analysis: ast.parse() constructs an Abstract Syntax Tree.

Phase 3 — Semantic analysis: scoped symbol-table pass records definitions and checks unresolved names.

Phase 4 — Metrics computation: AST traversal calculates complexity and nesting; source lines and comment tokens support LOC and comment density.

Phase 5 — Report generation: prints the report and writes JSON beside the analyzed sample.

Language: Python. Demonstration environment: Visual Studio Code with Windows PowerShell. The analyzer does not execute the input program.

4. How to Run the Project

Run these commands from the project folder containing analyzer.py:

```
python --version
dir samples
python analyzer.py samples/example1_classify.py
python analyzer.py samples/example2_process.py
python analyzer.py samples/example3_validate_orders.py
```

Each run prints a report and writes a .metrics.json file next to the sample.

5. Results

Metric	Example 1 classify	Example 2 process	Example 3 validate_orders
Total tokens	24	29	96
LOC	6	7	20
Cyclomatic complexity	3	3	7
Maximum nesting depth	3	3	5
Function count	1	1	1
Comment density	0.0%	14.3%	5.0%
Symbol-table entries	2	4	8
Semantic warnings	0	0	1

Example 3 token counts: 13 keywords, 30 identifiers, 43 operators, 9 literals, 1 comment, and 0 other tokens (96 total). LOC 20; cyclomatic complexity 7; maximum nesting depth 5; one function; comment density 5.0%.

6. Semantic Analysis Demonstration

Example 3 intentionally references unknown_id without defining it in the enclosing scope. The analyzer reports this unresolved name at line 19. This is an expected test warning demonstrating semantic analysis.

Observed warning: line 19: 'unknown_id' referenced but not found in any enclosing scope's symbol table.

Generated report: samples/example3_validate_orders.py.metrics.json.

7. Demonstration Screenshots

Figure 1. Example 1 — classify sample analysis.

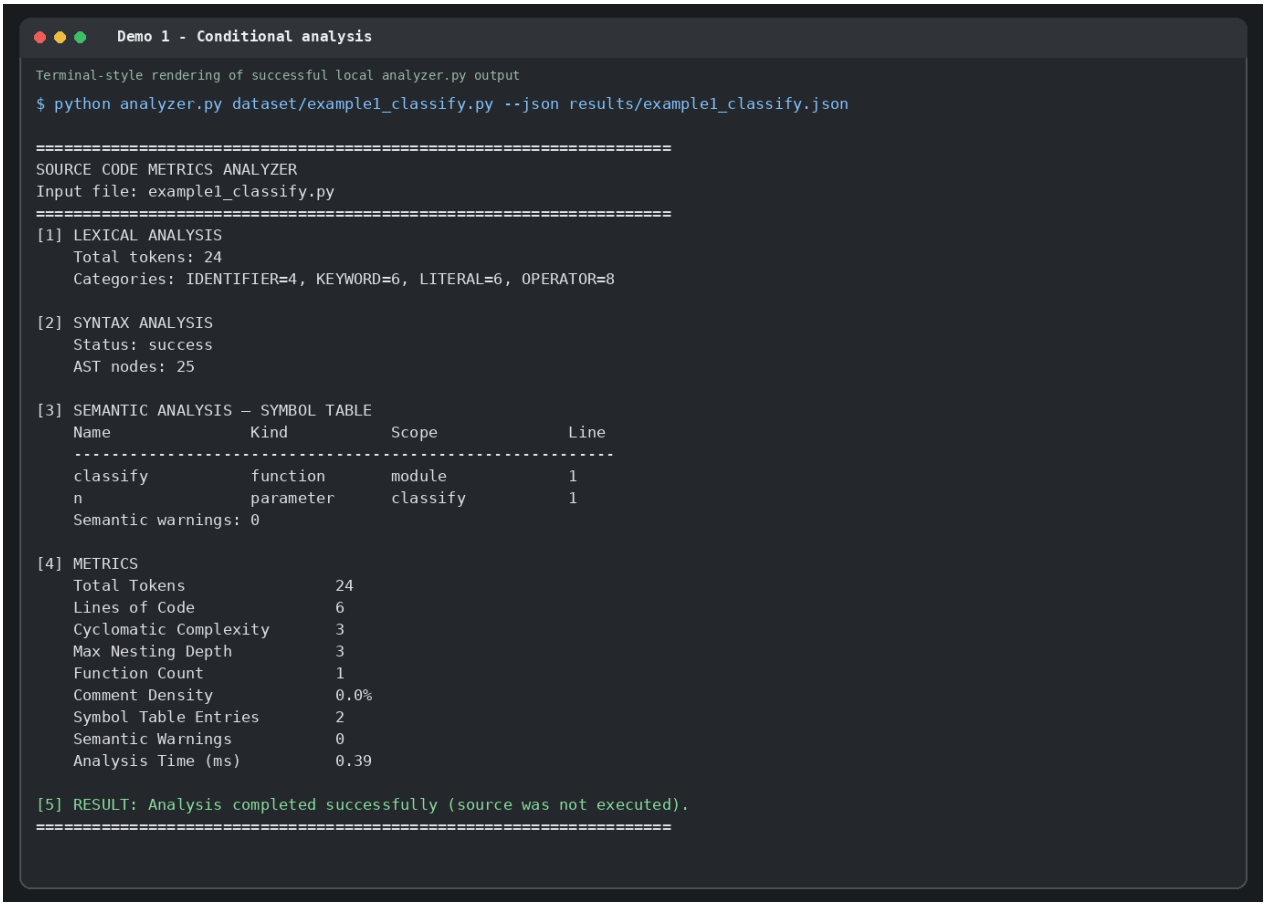


Figure 2. Example 2 — process sample analysis.

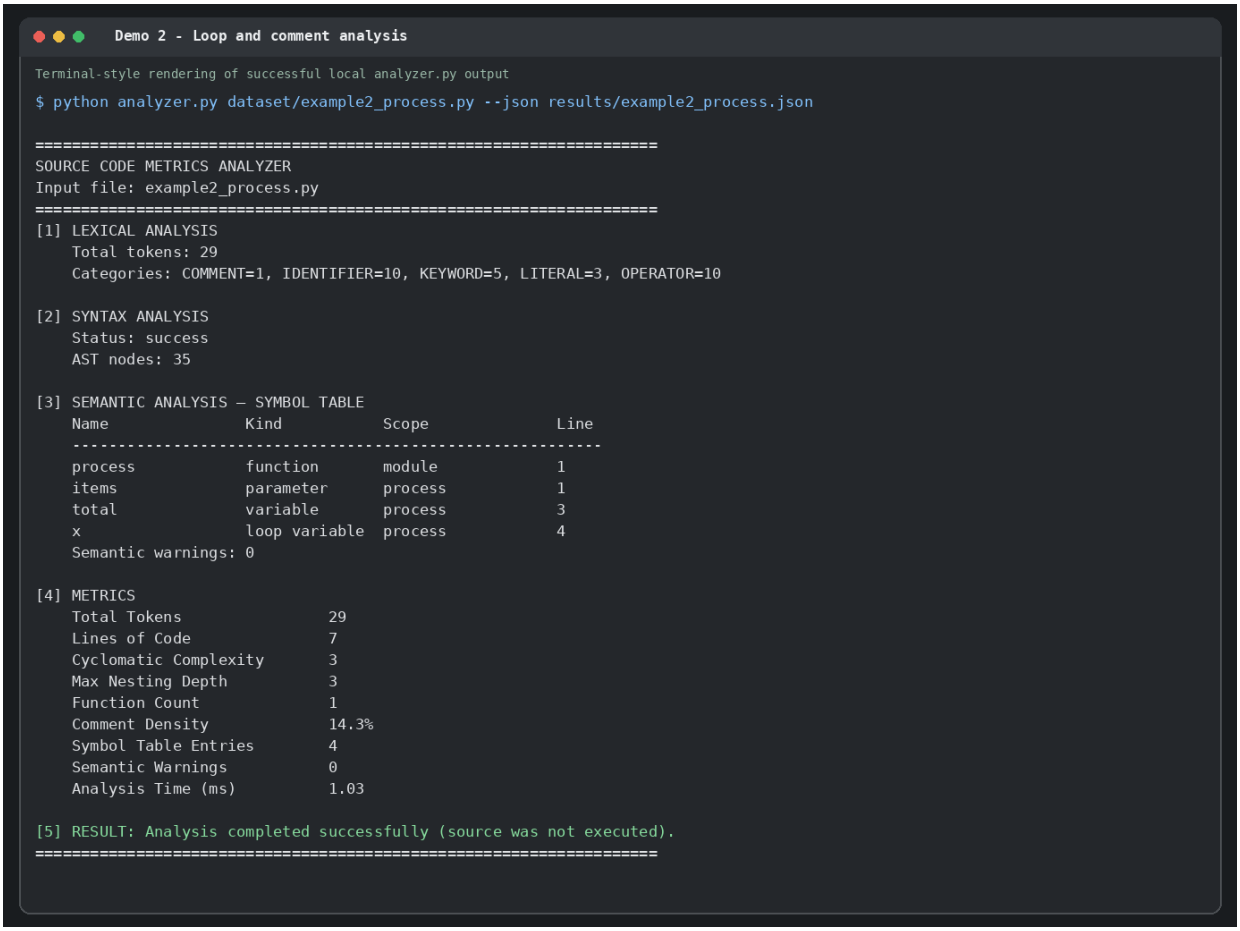
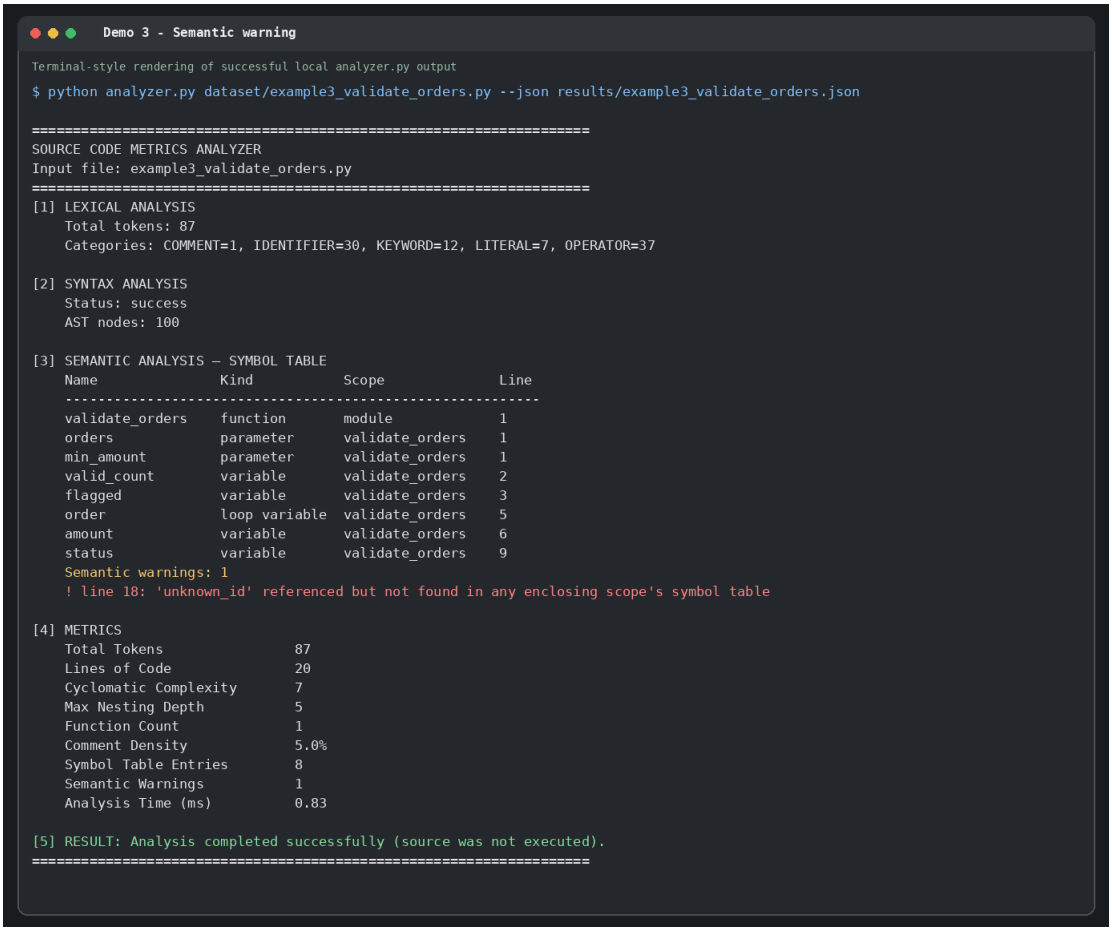


Figure 3. Example 3 — validate_orders analysis, semantic warning and metrics.



8. Conclusion and Limitations

The project demonstrates a static analysis pipeline connecting tokenization, AST construction, scoped symbol analysis, and metrics reporting. All three examples were run; the third demonstrates detection of an unresolved identifier. The analyzer targets Python and performs basic name-resolution checks rather than comprehensive type checking or cross-module analysis.

9. GitHub Repository

Public repository URL: [PASTE YOUR PUBLIC GITHUB REPOSITORY URL HERE AFTER UPLOADING]

The repository should contain analyzer.py, all three sample programs, generated metrics JSON files, demo screenshots, and this Review 3 document. A real URL must be inserted after creating and pushing the repository under the student's account.

10. Submission Checklist

■ Source code (analyzer.py)

■ Sample code / dataset

■ Results and JSON reports

■ Demo screenshots

■ Review 3 document

■ Public GitHub URL inserted

Appendix A — Complete Analyzer Source Code

Complete analyzer.py source used in the project package. Include the same file separately in GitHub.

```
#!/usr/bin/env python3
"""Source Code Metrics Analyzer - educational Python static analyzer.
Pipeline: tokenize -> AST parse -> scoped symbol table/name checks -> metrics -> JSON report.
The analyzer does not execute the analyzed source file.
"""
from __future__ import annotations
import argparse
import ast
import builtins
import io
import json
import keyword
import sys
import time
import tokenize
from collections import Counter
from pathlib import Path
from typing import Any
IGNORED_TOKEN_TYPES = {
    tokenize.ENCODING, tokenize.ENDMARKER, tokenize.NL, tokenize.NEWLINE,
    tokenize.INDENT, tokenize.DEDENT,
}
def lexical_analysis(source: str) -> dict[str, Any]:
    counts: Counter[str] = Counter()
    tokens: list[dict[str, Any]] = []
    generator = tokenize.generate_tokens(io.StringIO(source).readline)
    for tok in generator:
        if tok.type in IGNORED_TOKEN_TYPES:
            continue
        if tok.type == tokenize.COMMENT:
            category = "COMMENT"
        elif tok.type == tokenize.NAME and keyword.iskeyword(tok.string):
            category = "KEYWORD"
        elif tok.type == tokenize.NAME:
            category = "IDENTIFIER"
        elif tok.type in (tokenize.NUMBER, tokenize.STRING):
            category = "LITERAL"
        elif tok.type == tokenize.OP:
            category = "OPERATOR"
        else:
            category = "OTHER"
        counts[category] += 1
        tokens.append({"value": tok.string, "category": category, "line": tok.start[0]})
    return {"total_tokens": len(tokens), "category_counts": dict(sorted(counts.items())), "tokens": tokens}
def is_scope_boundary(node: ast.AST) -> bool:
    return isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef, ast.Lambda, ast.ClassDef))
def collect_scope_definitions(node: ast.AST) -> list[dict[str, Any]]:
    """Collect definitions belonging to one scope, without descending into nested scopes."""
    entries: list[dict[str, Any]] = []
    seen: set[tuple[str, str]] = set()
    def add(name: str, kind: str, line: int) -> None:
        key = (name, kind)
        if key not in seen:
            seen.add(key)
            entries.append({"name": name, "kind": kind, "line": line})
    if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
        add(node.name, "function", node.lineno)
        for arg in (*node.args.posonlyargs, *node.args.args, *node.args.kwonlyargs):
            add(arg.arg, "parameter", arg.lineno)
        if node.args.vararg:
            add(node.args.vararg.arg, "parameter", node.args.vararg.lineno)
        if node.args.kwarg:
            add(node.args.kwarg.arg, "parameter", node.args.kwarg.lineno)
    elif isinstance(node, ast.ClassDef):
        add(node.name, "class", node.lineno)
    def walk(current: ast.AST, root: bool = False) -> None:
        for child in ast.iter_child_nodes(current):
            if is_scope_boundary(child):
                # The nested function/class name is defined here, but its contents are separate.
                if isinstance(child, (ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)):
                    add(child.name, "function" if not isinstance(child, ast.ClassDef) else "class", child.lineno)
                continue
            if isinstance(child, ast.Name) and isinstance(child.ctx, ast.Store):
                kind = "loop variable" if parent_is_loop_target(current, child) else "variable"
                add(child.id, kind, child.lineno)
            elif isinstance(child, (ast.Import, ast.ImportFrom)):
                for alias in child.names:
                    add(alias.asname or alias.name.split('.')[0], "import", child.lineno)
            walk(child, root=False)
    walk(node, root=True)
    return entries
def parent_is_loop_target(parent: ast.AST, name_node: ast.Name) -> bool:
    if isinstance(parent, (ast.For, ast.AsyncFor)) and parent.target is name_node:
        return True
    if isinstance(parent, (ast.comprehension,)) and parent.target is name_node:
        return True
    return False
class LoadCollector(ast.NodeVisitor):
    def __init__(self) -> None:
        self.loads: list[tuple[str, int]] = []
    def visit_Name(self, node: ast.Name) -> None:
        if isinstance(node.ctx, ast.Load):
            self.loads.append((node.id, node.lineno))
    def visit_FunctionDef(self, node: ast.FunctionDef) -> None:
        for decorator in node.decorator_list:
            self.visit(decorator)
        # Function body is visited in its own scope by semantic_analysis.
        for default in node.args.defaults + node.args.kw_defaults:
            if default is not None:
                self.visit(default)
    visit_AsyncFunctionDef = visit_FunctionDef
    def visit_Lambda(self, node: ast.Lambda) -> None:
        for default in node.args.defaults + node.args.kw_defaults:
            if default is not None:
                self.visit(default)
    def visit_ClassDef(self, node: ast.ClassDef) -> None:
```

```
        for item in node.bases + node.decorator_list:
            self.visit(item)
def semantic_analysis(tree: ast.AST) -> dict[str, Any]:
    symbols: list[dict[str, Any]] = []
    warnings: list[dict[str, Any]] = []
    builtin_names = set(dir(builtins))
    # This educational symbol table summarizes module and top-level function scopes.
    module_entries = collect_scope_definitions(tree)
    module_names = {x["name"] for x in module_entries}
    for entry in module_entries:
        entry["scope"] = "module"
        symbols.append(entry)
    # Walk each function independently, checking names against locals and module/builtins.
    for node in ast.walk(tree):
        if not isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef, ast.Lambda)):
            continue
        entries = collect_scope_definitions(node)
        # Function's own name is in module scope; avoid duplicate in function scope.
        local_entries = [e for e in entries if e["kind"] != "function" or e["name"] != getattr(node, "name", None)]
        scope_name = getattr(node, "name", "<lambda>")
        local_names = {e["name"] for e in local_entries}
        for entry in local_entries:
            symbols.append({**entry, "scope": scope_name})
        collector = LoadCollector()
        # Traverse statements only; collector deliberately avoids nested function bodies.
        for statement in getattr(node, "body", []) if isinstance(getattr(node, "body", None), list) else [node.body]:
            collector.visit(statement)
        for name, line in collector.loads:
            if name not in local_names and name not in module_names and name not in builtin_names:
                warnings.append({"line": line, "name": name, "message": f"'{name}' referenced but not found in any enclosing scope's symbol table"})
    # Deduplicate warning produced by AST-level traversal when nested scopes are encountered.
    unique_warn = {(w["line"], w["name"]): w for w in warnings}
    return {"symbol_table": symbols, "symbol_table_entries": len(symbols), "semantic_warnings": list(unique_warn.values()), "semantic_warning_count": len(uni
class ComplexityVisitor(ast.NodeVisitor):
    DECISIONS = (ast.If, ast.For, ast.AsyncFor, ast.While, ast.IfExp, ast.Assert)
    def __init__(self) -> None:
        self.complexity = 1
    def generic_visit(self, node: ast.AST) -> None:
        if isinstance(node, self.DECISIONS):
            self.complexity += 1
        elif isinstance(node, ast.ExceptHandler):
            self.complexity += 1
        elif isinstance(node, ast.BoolOp):
            self.complexity += max(0, len(node.values) - 1)
        super().generic_visit(node)
def max_nesting_depth(tree: ast.AST) -> int:
    compound = (ast.If, ast.For, ast.AsyncFor, ast.While, ast.Try, ast.With,
                ast.AsyncWith, ast.FunctionDef, ast.AsyncFunctionDef, ast.ClassDef)
    max_depth = 0
    def walk(node: ast.AST, depth: int) -> None:
        nonlocal max_depth
        if isinstance(node, compound):
            depth += 1
            max_depth = max(max_depth, depth)
        for child in ast.iter_child_nodes(node):
            walk(child, depth)
    walk(tree, 0)
    return max_depth
def compute_metrics(source: str, tree: ast.AST, lexical: dict[str, Any], semantic: dict[str, Any], elapsed_ms: float) -> dict[str, Any]:
    lines = source.splitlines()
    loc = sum(1 for line in lines if line.strip())
    comments = lexical["category_counts"].get("COMMENT", 0)
    complexity = ComplexityVisitor()
    complexity.visit(tree)
    function_count = sum(1 for n in ast.walk(tree) if isinstance(n, (ast.FunctionDef, ast.AsyncFunctionDef)))
    return {
        "total_tokens": lexical["total_tokens"],
        "lines_of_code": loc,
        "cyclomatic_complexity": complexity.complexity,
        "max_nesting_depth": max_nesting_depth(tree),
        "function_count": function_count,
        "comment_count": comments,
        "comment_density_percent": round((comments / loc * 100) if loc else 0.0, 1),
        "symbol_table_entries": semantic["symbol_table_entries"],
        "semantic_warning_count": semantic["semantic_warning_count"],
        "analysis_time_ms": round(elapsed_ms, 2),
    }
def analyze_file(path: Path) -> dict[str, Any]:
    source = path.read_text(encoding="utf-8")
    start = time.perf_counter()
    lexical = lexical_analysis(source)
    try:
        tree = ast.parse(source, filename=str(path))
    except SyntaxError as exc:
        raise ValueError(f"Syntax analysis failed at line {exc.lineno}: {exc.msg}") from exc
    semantic = semantic_analysis(tree)
    elapsed_ms = (time.perf_counter() - start) * 1000
    metrics = compute_metrics(source, tree, lexical, semantic, elapsed_ms)
    return {
        "file": path.name,
        "pipeline": ["Lexical Analysis", "Syntax Analysis (AST)", "Semantic Analysis (Symbol Table)", "Metrics Computation", "JSON Report Generation"],
        "lexical_analysis": {"total_tokens": lexical["total_tokens"], "category_counts": lexical["category_counts"], "tokens": lexical["tokens"]},
        "syntax_analysis": {"status": "success", "ast_node_count": sum(1 for _ in ast.walk(tree))},
        "semantic_analysis": semantic,
        "metrics": metrics,
        "note": "Static analysis only: the input source file is parsed, not executed. Semantic checks are educational name-resolution checks, not full Python
    }
def print_report(report: dict[str, Any], show_tokens: bool = False, show_ast: bool = False, source_path: Path | None = None) -> None:
    print("=" * 68)
    print("SOURCE CODE METRICS ANALYZER")
    print(f"Input file: {report['file']}")
    print("=" * 68)
    print("[1] LEXICAL ANALYSIS")
    print(f"    Total tokens: {report['lexical_analysis']['total_tokens']}")
    print("    Categories: " + ", ".join(f"{k}={v}" for k, v in report['lexical_analysis']['category_counts'].items()))
    if show_tokens:
        for token in report["lexical_analysis"]["tokens"]:
            print(f"    line {token['line']:>3}: {token['category']:<10} {token['value']!r}")
    print("\n[2] SYNTAX ANALYSIS")
    print(f"    Status: {report['syntax_analysis']['status']}")
    print(f"    AST nodes: {report['syntax_analysis']['ast_node_count']}")
    if show_ast:
```

```
        print("    AST preview:")
        print("\n".join("    " + line for line in ast.dump(ast.parse((source_path or Path(report['file'])).read_text(encoding='utf-8')), indent=2).splitlines
    print("\n[3] SEMANTIC ANALYSIS — SYMBOL TABLE")
    print(f"    {'Name':<18} {'Kind':<14} {'Scope':<18} Line")
    print("    " + "-" * 58)
    for s in report["semantic_analysis"]["symbol_table"]:
        print(f"    {s['name']:<18} {s['kind']:<14} {s['scope']:<18} {s['line']}")
    warnings = report["semantic_analysis"]["semantic_warnings"]
    print(f"    Semantic warnings: {len(warnings)}")
    for w in warnings:
        print(f"    ! line {w['line']}: {w['message']}")
    print("\n[4] METRICS")
    labels = [
        ("Total Tokens", "total_tokens"), ("Lines of Code", "lines_of_code"),
        ("Cyclomatic Complexity", "cyclomatic_complexity"), ("Max Nesting Depth", "max_nesting_depth"),
        ("Function Count", "function_count"), ("Comment Density", "comment_density_percent"),
        ("Symbol Table Entries", "symbol_table_entries"), ("Semantic Warnings", "semantic_warning_count"),
        ("Analysis Time (ms)", "analysis_time_ms")]
    for label, key in labels:
        value = report["metrics"][key]
        if key == "comment_density_percent":
            value = f"{value:.1f}%"
        print(f"    {label:<27} {value}")
    print("\n[5] RESULT: Analysis completed successfully (source was not executed).")
    print("=" * 68)
def main() -> int:
    parser = argparse.ArgumentParser(description="Statically analyze Python source code and report software metrics.")
    parser.add_argument("source", type=Path, help="Path to a .py source file to analyze")
    parser.add_argument("--json", dest="json_path", type=Path, help="Write the complete analysis report to this JSON file")
    parser.add_argument("--tokens", action="store_true", help="Print every classified token")
    parser.add_argument("--ast", action="store_true", help="Print a short AST preview")
    args = parser.parse_args()
    if not args.source.is_file():
        print(f"ERROR: input file not found: {args.source}", file=sys.stderr)
        return 2
    try:
        report = analyze_file(args.source)
    except (OSError, UnicodeError, ValueError, tokenize.TokenError) as exc:
        print(f"ERROR: {exc}", file=sys.stderr)
        return 1
    print_report(report, show_tokens=args.tokens, show_ast=args.ast, source_path=args.source)
    if args.json_path:
        args.json_path.parent.mkdir(parents=True, exist_ok=True)
        args.json_path.write_text(json.dumps(report, indent=2), encoding="utf-8")
        print(f"\nJSON report saved to: {args.json_path}")
    return 0
if __name__ == "__main__":
    raise SystemExit(main())
```

Appendix B — Sample Input: example1_classify.py

```
def classify(n):
    if n > 0:
        if n % 2 == 0:
            return "pos-even"
        return "pos-odd"
    return "non-positive"
```

Appendix B — Sample Input: example2_process.py

```
def process(items):
    # sum of even squares
    total = 0
    for x in items:
        if x % 2 == 0:
            total += x * x
    return total
```

Appendix B — Sample Input: example3_validate_orders.py

```
def validate_orders(orders, min_amount):
    valid_count = 0
    flagged = []
    # Validate each order before counting it
    for order in orders:
        amount = order.get("amount", 0)
        if amount >= min_amount:
            try:
                status = order["status"]
                while status == "pending":
                    valid_count += 1
                    break
                assert amount >= 0
            except KeyError:
                flagged.append(order)
        else:
            flagged.append(order)
    if unknown_id:
        flagged.append(order)
    return valid_count, flagged
```